

CineReads

A Movie & Book Search Engine Built on an Inverted Index with TF-IDF Ranking

Information Retrieval / Search Engine — Project Report

MD Tanveer Mahmood Shanin (ID: 0432220005101117) Joydev Datta (ID: 0432220005101132)

Course: CSE 426 — Information Retrieval

University of Information Technology and Sciences (UITS), Dhaka

GitHub: <https://github.com/ShaninX48>

1. Problem Definition & Dataset

1.1 Domain

CineReads is a search engine over a combined catalogue of **movies and books**. The user types a free-text query — a title fragment, an author or director, a genre, or a thematic phrase — and the system returns the most relevant titles, ranked, from a single unified index. The motivation is that people often look for entertainment across both media (“give me space adventures”, “crime thrillers”) and want one place to search both, with the more relevant and better-regarded titles surfaced first.

1.2 Dataset

Two public Kaggle datasets are merged into one document collection:

- **Movies** — “The Movies Dataset” ([rounakbanik/the-movies-dataset](#)). We read `movies_metadata.csv` and keep the first 10,000 rows. Useful fields: title, overview, genres (stored as a stringified list of dicts), `vote_average`, release date, original language.
- **Books** — “Books Dataset: 15k Books across 100 Categories” ([mihikaajayjadhav/books-...](#)). Fields: title, author, category, description, rating, year, number of ratings.

After parsing, every record becomes a uniform document dictionary with a shared schema (`id`, `type`, `title`, `rating`, `year`) plus type-specific fields, so the index never has to special-case movies versus books. The final corpus:

Metric	Value
Total documents (N)	25,139
Movies	10,000
Books	15,139
Unique index terms (after stemming)	49,993
Average document frequency per term	18.8
Most frequent term (df)	“drama” — 5,484 documents

Table 1. Corpus statistics, read directly from the built index.

1.3 Why this is a non-routine problem (P4)

Building this engine is not a matter of calling a library function; several genuine challenges had to be solved:

- **Heterogeneous sources.** Movies and books have different columns, scales, and conventions (movie ratings are out of 10, book ratings out of 5; genres are encoded differently). They must be normalised into one comparable representation before they can share an index and a ranking function.
- **Messy real-world CSVs.** Column names vary between dataset versions, some rows are malformed, and the movie `genres` field is a Python-literal string that must be safely parsed. The loader has to be defensive (alias lookup, bad-line skipping, NaN handling) rather than assume clean input.
- **Scale.** Indexing ~25k documents and ~50k terms naively (e.g. scanning every document per query) would be too slow; the inverted index and set-based Boolean retrieval exist precisely to avoid linear scans.

- **Ranking across media.** A single relevance score has to compare a book and a movie fairly, which forces explicit decisions about normalisation and about how much non-textual signal (ratings) should influence the order.

2. Preprocessing & Inverted Index Design

2.1 Preprocessing pipeline

The same `preprocess()` function is applied both to documents at index time and to queries at search time. Using one pipeline for both guarantees that query terms and indexed terms live in exactly the same vocabulary (e.g. both “adventures” and “adventure” collapse to the stem `adventur`). The steps:

- **Lowercasing** — case-insensitive matching.
- **Punctuation removal** — every non-alphanumeric character is replaced by a space via a regular expression.
- **Tokenisation** — split on whitespace.
- **Stopword removal** — common words (the, of, and, ...) and single-character tokens are dropped using NLTK’s English stopwords list, reducing index size and noise.
- **Porter stemming** — words are reduced to their root (`romance`, `romantic` → `romanc`), so morphological variants match.

```
def preprocess(self, text):
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s]', ' ', text)    # strip punctuation
    tokens = text.split()
    if self.remove_stopwords:
        tokens = [t for t in tokens if t not in self.stop_words and len(t) > 1]
    if self.use_stemming:
        tokens = [self.stemmer.stem(t) for t in tokens]
    return tokens
```

2.2 Inverted index structure

The core data structure maps each term to the documents that contain it, and within each document to the list of token positions:

```
index[term][doc_id] = [pos1, pos2, ...]    # positional postings list
term_df[term]       = number of documents containing the term
doc_store[doc_id]   = the original document (for display)
doc_lengths[doc_id] = token count of the document
total_docs          = N (used for IDF)
```

During indexing, each document’s searchable fields (title, author/director, description/overview, category/genres) are concatenated, preprocessed, and every token is appended to the postings list for its term. A per-document `seen` set ensures each term contributes to `term_df` only once per document, which keeps document frequency correct even for repeated words.

2.3 Justifying the design choices (P1)

- **Why an inverted index?** It turns search from an $O(N)$ scan of all 25,139 documents into a lookup of a handful of short postings lists — the standard, scalable structure behind every real search engine.

- **Why positional postings (storing positions, not just counts)?** Positions cost more memory but enable the relevance-aware snippet generator today, and make future phrase/proximity search possible without re-indexing.
- **Why stemming + stopwords removal?** Together they shrink the vocabulary and raise recall: a query for “adventures” still matches a document titled “Adventure”. The trade-off is a small loss of precision (stemming can conflate unrelated words), judged worthwhile for this catalogue.
- **Why pickle the index?** Building the index over 25k documents takes time and a Kaggle download; caching it to `search_index.pkl` makes every subsequent launch start in well under a second.

3. Query Processing & Ranking

3.1 Query processing

A query is handled in four stages: (1) it is run through the same preprocessing pipeline; (2) a **candidate set** of documents is selected using Boolean retrieval; (3) the candidates are optionally filtered by type; (4) the candidates are scored and sorted, and the top-k are returned with a snippet and the list of matched terms.

Boolean retrieval operates directly on postings sets:

- **AND mode** returns the intersection of the postings sets — documents containing *every* query term. Precise, but empty if any term is missing.
- **OR mode** returns the union — documents containing *any* query term. Higher recall, useful as a fallback when AND returns nothing.

```
def boolean_and(self, terms):
    result = self._postings(terms[0])
    for t in terms[1:]:
        result &= self._postings(t)    # set intersection
    return result

def boolean_or(self, terms):
    result = set()
    for t in terms:
        result |= self._postings(t)    # set union
    return result
```

3.2 TF-IDF ranking

Candidates are ranked by **TF-IDF**, the classic weighting that rewards terms which are frequent in a document but rare across the corpus:

```
TF(t, d) = 1 + log(count(t, d))    # log-normalised; 0 if absent
IDF(t)   = log(N / df(t))          # N = 25,139
tfidf    = TF(t, d) * IDF(t)

score(d) = sum( tfidf(t, d) for t in query ) + 0.3 * (rating / max_rating)
```

Log-normalised TF prevents a term that appears 50 times from dominating linearly. **IDF** down-weights ubiquitous terms (“drama”, “book”) and rewards distinctive ones. A small **rating boost** (capped at 0.3) acts only as a tie-breaker so that, among textually similar matches, higher-rated titles rank first; ratings are normalised by each medium’s own scale (10 for movies, 5 for books) so the comparison is fair.

3.3 Trade-offs: accuracy vs. efficiency

- **Boolean candidate selection before scoring.** Rather than scoring all 25k documents, the engine only scores the (usually small) Boolean candidate set. This is a large efficiency win; the accuracy cost is that AND mode can miss partially-matching documents — mitigated by offering OR mode.
- **Snippet windowing.** The snippet generator slides a 30-word window over the description to find the densest cluster of query terms. This noticeably improves preview quality but is the most expensive per-result step; it is therefore applied only to the final top-k, not to every candidate.
- **No length normalisation in TF-IDF.** The score does not divide by document length, which is simpler and faster but can mildly favour longer descriptions. Cosine/BM25-style length normalisation is listed as future work.
- **Rating boost weight.** Set deliberately low (0.3). A larger weight would surface popular titles faster but risk burying a textually perfect match under a merely popular one.

4. System Analysis & Discussion

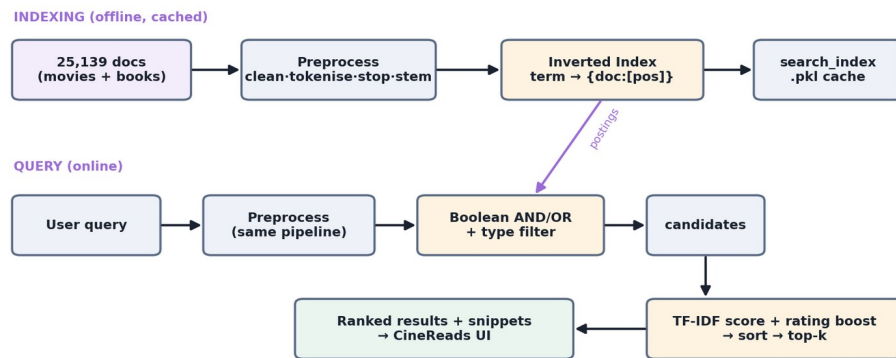


Figure 1. CineReads architecture: an offline indexing path (cached to disk) and an online query path.

4.1 Key design decisions

- **Separation of concerns.** The IR engine (`inverted_index.py`) has no web dependency; the Flask layer (`server.py`) and data loading (`data_loader.py`) are isolated. This makes the engine independently testable and the system easy to reason about.
- **Unified document schema.** Normalising both media into one dict shape means a single index, a single scoring function, and a single results template handle everything.
- **Cache-first start-up.** `run.py` loads `search_index.pkl` if present and only downloads/rebuilds when it is missing.
- **Thin, RESTful API.** Four small JSON endpoints (`/api/search`, `/api/stats`, `/api/suggest`, and `/`) keep the front-end and back-end loosely coupled.

4.2 Challenges faced

- **Parsing the movie genres field,** stored as a stringified list of dictionaries, required `ast.literal_eval` with a safe fallback.
- **Inconsistent column names** across dataset versions were handled with an alias-lookup helper (`find_col`).

- **Matching query and document vocabularies** — early versions stemmed documents but not queries, so queries silently failed; unifying the pipeline fixed this.

4.3 Reasoning and possible improvements (P3)

The current design optimises for clarity and correctness on a mid-sized catalogue. Several improvements would raise quality further:

- **BM25 ranking** with document-length normalisation would handle varying description lengths better than plain TF-IDF.
- **Phrase and proximity search** — the positional postings are already stored, so matching exact phrases (“science fiction”) is a natural and cheap extension.
- **Spell-correction / fuzzy matching** (e.g. edit distance on the term dictionary) would stop misspelled AND-queries returning nothing.
- **Enriching movie metadata** by joining `credits.csv` to populate the currently-empty director field.
- **Field weighting** — a title match should arguably count more than a description match; per-field boosts would improve precision.

5. Results & Conclusion

5.1 Sample outputs

The following results are real output from the built index. For the query “**space adventure**” in AND mode (preprocessed to the stems `space`, `adventur`), the top results are:

#	Type	Title	Score
1	Book	The Good Old Stuff	14.13
2	Movie	Lost in Space	12.13
3	Book	Let’s Go On An Adventure	11.73
4	Book	Hiking Camping Journal: Wilderness Adventures	11.73

Table 2. Top results for “space adventure” (AND mode, all types).

For “**vampire romance**” (AND), the engine correctly mixes both media, with the movie **Vampire in Brooklyn** appearing alongside vampire-romance books; for “**crime thriller**” (AND) the top hit is a book whose title literally contains “Crime Thriller”, scoring far above the rest (27.7) because both rare terms co-occur in a short, high-signal title.

Query (mode)	Top result	Score
crime thriller (AND)	Searching for Faith (A Carissa Jones Crime Thriller)	27.71
vampire romance (AND)	Twice As High	26.10
love war (OR)	Love Stories of World War II	16.69

Table 3. Top result for three further queries (real index output).

5.2 Findings

- The inverted index makes retrieval effectively instant on 25k documents: queries touch only the relevant postings lists rather than scanning the corpus.
- TF-IDF produces intuitive rankings — titles where both query terms are rare and co-occur (e.g. “crime thriller”) score markedly higher than partial matches.
- A practical data limitation surfaced during testing: **most book ratings in the dataset are 0**, so the rating boost currently influences movie ordering far more than book ordering. This is a dataset issue, not a ranking bug, and is documented as future work.

5.3 Conclusion

CineReads demonstrates a complete, from-scratch information-retrieval pipeline: heterogeneous data is cleaned and unified, indexed into a positional inverted index of ~50k terms, and queried through Boolean retrieval with TF-IDF ranking, all behind a responsive web UI. The project meets its goals of correctness, reasonable efficiency, and a clear, well-separated design, while leaving a well-defined path — BM25, phrase search, spell-correction, and richer metadata — for future improvement.